

Internal Working of remove() in Python List

Before understanding this method

A Python list works like a dynamic array. Internally it has two important ideas:

- **size** - number of elements currently present in the list.
- **capacity** - extra internal space reserved by Python for future elements.

When capacity is full, Python creates a bigger internal array, copies old elements, and then performs the operation.

Meaning

remove() removes the first occurrence of a given value from the list.

```
a = [10, 20, 30, 20, 40]
a.remove(20)
print(a)    # [10, 30, 20, 40]
```

Internal steps

- Python starts searching from index 0.
- It compares each element with the value to be removed.
- When the first matching value is found, Python removes it.
- Elements after the removed value are shifted one position to the left.
- The size of the list decreases by 1.

Before remove(20):

```
Index:  0   1   2   3   4
List:  10  20  30  20  40
```

Search starts:

```
Index 0 -> 10, not equal to 20
Index 1 -> 20, found
```

Remove first 20 and shift left:

```
30 moves to index 1
20 moves to index 2
40 moves to index 3
```

Final list:

```
[10, 30, 20, 40]
```

If value is not found

```
a = [10, 20, 30]
a.remove(50)
# ValueError: list.remove(x): x not in list
```

Safe way to use remove()

```
a = [10, 20, 30]
```

```
if 50 in a:
    a.remove(50)
```

```
else:  
    print("Value not found")
```

Time complexity

- Searching can take $O(n)$.
- Shifting after removal can also take $O(n)$.
- Overall worst-case time complexity is $O(n)$.

Key point

`remove()` removes by value, not by index, and only removes the first matching value.